

Jetson Orin Nano Configuration

System Requirements

Host Machine (for SDK Manager)

- OS: Ubuntu 22.04 LTS (recommended for SDK Manager)
- Architecture: x86_64
- Internet connection: Required
- RAM: Minimum 8 GB (16 GB recommended)
- Storage: Minimum 43 GB free space (64 GB+ recommended)
- USB cable for Jetson recovery mode

Target Device

- NVIDIA Jetson Orin Nano Developer Kit
- microSD card or NVMe SSD
- Power supply

Installation

Prepare Host Machine

1. Boot a PC/laptop with Ubuntu 22.04 LTS.
2. Install NVIDIA SDK Manager.
3. Connect the Jetson Orin Nano using USB.
4. Put the Jetson into Force Recovery Mode.

Install SDK Manager

Download and install SDK Manager:

```
wget
https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64/
cuda-keyring_1.1-1_all.deb
sudo dpkg -i cuda-keyring_1.1-1_all.deb
sudo apt update
sudo apt install sdkmanager# Jetson Nano Configuration
```

Launch:

```
sdkmanager
```

Flash JetPack SDK

In SDK Manager:

1. Log in with NVIDIA account
2. Select:

```
Product Category:
Jetson

Hardware:
Jetson Orin Nano Developer Kit

JetPack:
JetPack 7.x
```

3. Select Target Storage:

- MicroSD Card
- NVMe SSD

4. Start installing

SDK Manager will install:

- Jetson Linux (L4T)
- Ubuntu 24.04 target system
- CUDA
- cuDNN
- TensorRT
- NVIDIA drivers
- Multimedia libraries

Verify Installation

On the Jetson:

Check JetPack:

```
cat /etc/nv_tegra_release
```

Check CUDA:

```
nvcc --version
```

Check GPU:

```
nvidia-smi
```

Environment Configuration

Target stack:

```
Jetson Orin Nano
├── JetPack 7
│   ├── NVIDIA Driver
│   ├── CUDA
│   ├── cuDNN
│   └── TensorRT
├── Docker + NVIDIA Container Runtime
└── YOLO Environment
    ├── PyTorch
    ├── Ultralytics
    ├── OpenCV
    └── Your SmartDB AI application
```

First boot

On the jetson:

```
sudo apt update
sudo apt upgrade -y
```

Install tools:

```
sudo apt install -y \
git \
curl \
wget \
vim \
htop \
python3-pip \
python3-venv \
build-essential
```

Install Jetson Monitoring tools

```
sudo pip3 install jetson-stats
```

Reboot:

```
sudo reboot
```

Run:

```
jtop # like htop but for jetson
```

Install NVIDIA container runtime

Docker is recommended for your YOLO environment.

Install Docker:

```
sudo apt install -y docker.io
```

Enable:

```
sudo systemctl enable docker  
sudo systemctl start docker
```

Add yourself:

```
sudo usermod -aG docker $USER
```

Logout/login.

Install NVIDIA container runtime:

```
sudo apt install -y nvidia-container
```

Test:

```
docker info
```

Install NVIDIA container toolkit

For Jetson:

```
sudo apt install nvidia-container
```

Restart Docker:

```
sudo systemctl restart docker
```

Test GPU access:

```
docker info | grep NVIDIA
```

Create YOLO workspace

```
mkdir ~/smartdb_ai  
cd ~/smartdb_ai
```

Structure:

```
smartdb_ai/  
├── models/  
│   └── best.pt  
├── src/  
│   ├── detector.py  
│   └── camera.py  
├── datasets/  
└── Dockerfile
```

Use an NVIDIA L4T PyTorch container.

Example:

```
docker run --runtime nvidia \  
--network host \  
--volume ~/smartdb_ai:/workspace \  
-it nvcr.io/nvidia/l4t-pytorch:r36.4.0
```

Inside container:

```
pip install ultralytics  
pip install opencv-python
```

Test:

```
import torch  
  
print(torch.cuda.is_available())
```

Expected:

```
True
```

Test YOLO

Inside container:

```
yolo predict \  
model=models/best.pt \  
source=0
```

or Python:

```
from ultralytics import YOLO  
  
model = YOLO("models/best.pt")  
  
results = model("test.jpg")  
  
for r in results:  
    print(r.bboxes)
```

Optimize for Jetson

For deployment, don't run `.pt` directly.

Convert:

```
from ultralytics import YOLO  
  
model = YOLO("best.pt")
```

```
model.export(  
    format="engine",  
    half=True  
)
```

Output:

```
best.engine
```

Then inference uses TensorRT:

```
Camera  
  ↓  
TensorRT YOLO  
  ↓  
Detection  
  ↓  
SmartDB logic
```

Camera setup

USB camera:

```
sudo apt install v4l-utils
```

Check:

```
v4l2-ctl --list-devices
```

SmartDB recommended architecture

For electrical fault detection project:

```
Sensors  
(PZEM / CT / Thermal Camera)  
  |  
  |  
  v  
ESP32 / Raspberry Pi  
  |  
  |  
  v
```

Jetson Orin Nano

